The Feedback Vertex Set Problem: Theory, Algorithms, and Applications

CSE472 — Algorithm Engineering Sessional

Project Report — Group 06

2005090 Tawkir Aziz Rahman

2005068 Suman Hossain

2005074 Dipanta Kumar Roy Nobo

2005091 Waseem Mustak Zisan

2005104 Md Hasin Arafat Al Sifat

2005109 Noushin Tabassum Aoishy

Department of Computer Science and Engineering

Bangladesh University of Engineering and Technology (BUET)

Dhaka, Bangladesh

April 2026

GitHub Repository:

https://github.com/TAR2003/Feedback_Vertex_Set_Problem_Algorithm_Project_4-2

Abstract

The Feedback Vertex Set (FVS) problem asks for a minimum set of vertices in a graph whose removal renders the graph acyclic. It is a fundamental combinatorial problem with applications in operating system deadlock prevention, circuit stabilization, and dependency resolution. While the decision version of FVS is classically NP-Complete, fixed-parameter tractable (FPT) algorithms exist parameterized by the solution size k, making it tractable for small k.

This report presents a comprehensive research and implementation framework addressing both the undirected FVS problem and the Directed Feedback Vertex Set (DFVS) problem. We implement three classes of solvers: *exact FPT solvers* (Bounded Search Tree and Iterative Compression), *heuristic evolutionary solvers* (Memetic Algorithm), and novel *hybrid ML-combinatorial solvers* (GNN-KMA and GNN-DKMA).

Our primary algorithmic contributions are: (i) the **Kernelized Memetic Algorithm (KMA)**, which integrates graph-theoretic reduction rules — including SCC-based directed reduction that eliminates 60–90% of vertices — with a C++-backed evolutionary search; (ii) the **Dynamic Kernelized Memetic Algorithm (DKMA)**, which interleaves population-based search with progressive commitment and re-kernelization, enabling an *opening-up* effect that standard static kernelization cannot exploit; and (iii) three generations of **GNN-KMA** models (v1, v2, v3), where graph neural networks predict high-confidence FVS vertices that are hard-fixed prior to KMA, under a precision-first coupling strategy that caps false-positive risk.

Experimentally, we generate and publicly release the largest labeled FVS dataset to our knowledge: 100K undirected and 100K directed graphs spanning five graph families and two difficulty tracks, with solver-verified ground-truth labels. On the PACE 2022 benchmark, our best configuration (KMA with population 90) achieves a mean score of 78.93 out of 100, while GNN-KMA v3 reaches 78.64, competitive with world-class solvers. All exact solvers achieve 100% optimality on their completed instances.

This work demonstrates that integrating kernelization, evolutionary search, and graph neural network guidance into a unified pipeline yields competitive solutions across both small exact instances and large heuristic-track graphs.

Keywords: Feedback Vertex Set, Directed Feedback Vertex Set, Parameterized Algorithms, Kernelization, Memetic Algorithm, Graph Neural Networks, Hybrid Solver, PACE 2022.

Contents

Abstract **1**

1 Introduction **7**
1.1 Background and Motivation . . . 7
1.2 Problem Statement . . . 8
1.3 Objectives of This Project . . . 8
1.4 Summary of Contributions . . . 8
1.5 Report Organization . . . 9

2 Theoretical Background **10**
2.1 Graph-Theoretic Foundations . . . 10
2.1.1 Core Observations . . . 10
2.1.2 Additional Visual Examples . . . 11
2.2 Problem Formulations . . . 11
2.3 Complexity-Theoretic Background . . . 11
2.3.1 Why FVS Is in NP . . . 12
2.3.2 NP-Hardness via Reduction from Vertex Cover . . . 12
2.3.3 Reverse Reduction Insight (FVS → VC) . . . 13
2.4 Directed Graph Perspective (SCC View) . . . 13

3 Related Work **14**
3.1 Overview . . . 14
3.2 Exact Exponential and Parameterized Algorithms . . . 14
3.2.1 Motivation . . . 14
3.2.2 Naive Brute Force . . . 14
3.2.3 Dynamic Programming on Subsets . . . 15
3.2.4 Iterative Compression . . . 15
3.2.5 Bounded Search Tree . . . 15
3.2.6 Advanced FPT Algorithms . . . 15
3.3 Approximation Algorithms . . . 16
3.3.1 Motivation . . . 16
3.3.2 2-Approximation Algorithm . . . 16
3.3.3 Logarithmic Approximation . . . 16
3.3.4 LP Relaxation and Rounding . . . 16
3.4 Randomized Algorithms . . . 16
3.4.1 Motivation . . . 16
3.4.2 Random Sampling . . . 17
3.4.3 Randomized Rounding . . . 17
3.4.4 Color Coding . . . 17
3.5 Heuristic Algorithms . . . 17
3.5.1 Motivation . . . 17

3.5.2 Greedy Vertex Selection . . . 17
3.5.3 Cycle-Based Greedy . . . 18
3.5.4 Core-Based Methods . . . 18
3.5.5 Reduction Rules + Greedy . . . 18
3.6 Metaheuristic Algorithms . . . 18
3.6.1 Motivation . . . 18
3.6.2 Genetic Algorithms . . . 18
3.6.3 Simulated Annealing . . . 19
3.6.4 Ant Colony Optimization . . . 19
3.6.5 Tabu Search . . . 19
3.6.6 Particle Swarm Optimization . . . 19
3.7 Summary and Positioning of Our Work . . . 19

4 Exact Algorithms **20**
4.1 Iterative Compression . . . 20
4.1.1 Core Idea . . . 20
4.1.2 Complexity Analysis . . . 20
4.1.3 Advantages and Limitations . . . 21
4.1.4 Simulation . . . 21
4.2 Bounded Search Tree (BST) . . . 22
4.2.1 Algorithm Description . . . 22
4.2.2 Complexity Analysis . . . 23
4.2.3 Simulation . . . 23

5 The Memetic Algorithm **25**
5.1 Algorithm Overview . . . 25
5.2 Encoding and Decoding . . . 25
5.3 Genetic Operators . . . 25
5.4 Complexity and Parameters . . . 26
5.4.1 Simulation . . . 26

6 KMA: Kernelized Memetic Algorithm **29**
6.1 Motivation . . . 29
6.2 KMA Pipeline . . . 29
6.3 Kernelization Rules . . . 29
6.3.1 Undirected Kernelization . . . 29
6.3.2 Directed Kernelization . . . 30
6.4 C++ Backend Solver . . . 30

7 DKMA: Dynamic Kernelized Memetic Algorithm **31**
7.1 Motivation: The Opening-Up Effect . . . 31
7.2 DKMA Algorithm . . . 31
7.3 Key Innovations . . . 32
7.3.1 Commit Threshold and Cap . . . 32
7.3.2 SHORTONE Directed Contractions . . . 32
7.3.3 Topological Diversification . . . 32
7.3.4 Gain Local Search . . . 32
7.3.5 Correctness Safeguards . . . 32

7.4 KMA vs. DKMA: Comparison . . . 33

8 GNN-KMA Models **34**
8.1 Motivation: Why Learn to Guide Search? . . . 34
8.2 Precision-First Coupling Strategy . . . 34
8.3 GNN-KMA v1: Baseline Architecture . . . 34
8.3.1 Feature Engineering (3 channels) . . . 35
8.3.2 Model Architecture . . . 35
8.3.3 Training . . . 35
8.4 GNN-KMA v2: Richer Structural Features . . . 35
8.5 GNN-KMA v3: Attention-Based Precision Model . . . 35
8.5.1 Feature Engineering (16 channels) . . . 36
8.5.2 Model Architecture: DirectedFVSNetV3 . . . 36
8.5.3 Training Innovations . . . 36
8.6 GNN-DKMA . . . 36
8.7 Architecture Evolution Summary . . . 37

9 Implementation Architecture **38**
9.1 System Architecture . . . 38
9.2 Data Structures and Key Abstractions . . . 38
9.3 C++ Engine Dispatch . . . 38
9.4 Reproducibility and Resume Semantics . . . 39
9.5 GNN Inference Integration . . . 39

10 Datasets and Experimental Setup **40**
10.1 Synthetic Dataset . . . 40
10.1.1 Design Goals . . . 40
10.1.2 Dataset Composition . . . 40
10.1.3 Track Size Intent . . . 41
10.1.4 Ground-Truth Labels . . . 41
10.1.5 Scale and Availability . . . 41
10.2 PACE 2022 Benchmark . . . 41
10.3 Evaluation Metrics . . . 41
10.4 Experimental Protocol . . . 42

11 Results and Evaluation **43**
11.1 Exact Solver Results . . . 43
11.1.1 Optimality on Completed Instances . . . 43
11.1.2 Scalability Analysis . . . 43
11.2 PACE 2022 Heuristic Results . . . 44
11.3 Comparative Analysis . . . 44
11.3.1 Kernelization Impact: MA vs. KMA . . . 44
11.3.2 Dynamic Kernelization: DKMA vs. KMA . . . 44
11.3.3 GNN Guidance: GNN-KMA vs. KMA . . . 44
11.3.4 Population Scaling . . . 44
11.4 Ablation: Feature and Architecture Contribution . . . 45

12 Discussion **46**

12.1 When Exact Algorithms Win . . . 46
12.2 When Metaheuristics Dominate . . . 46
12.3 Structural Graph Observations . . . 46
12.4 Practical Implications . . . 46
12.5 Limitations . . . 47

13 Future Work **48**

14 Conclusion **49**

A Directory Structure **52**

List of Figures

2.1 Basic FVS intuition: removing a vertex from every cycle makes the graph acyclic. . . . 10
2.2 From simple to harder structures: cycle interaction determines FVS size. . . . 11
2.3 Complex graph example where removing only node 5 makes the graph acyclic. . . . 11
2.4 Edge-to-triangle gadget used in the reduction Vertex Cover $\leq_p$ FVS. . . . 12
2.5 Full reduction illustration: each original edge is converted to a triangle gadget. Breaking all triangles corresponds to covering all original edges. . . . 12
2.6 Reverse Reduction Insight: FVS to VC gadget construction with triangle replacements and cross-edges. . . . 13

List of Tables

5.1 Default MA / KMA parameters . . . 26

7.1 Feature comparison between KMA and DKMA . . . 33

8.1 GNN-KMA v1/v2/v3 feature and architecture comparison . . . 37

10.1 Undirected graph categories and weights . . . 40
10.2 Directed graph categories and weights . . . 40
10.3 Track-specific graph size ranges . . . 41

11.1 Exact solver performance on undirected exact-track instances 43
11.2 PACE 2022 benchmark results (200 instances, 600s total) 44

Chapter 1

Introduction

1.1 Background and Motivation

Graphs are one of the most fundamental structures in computer science, providing a natural abstraction for networks ranging from the internet to social relationships, circuit topologies, and program dependency graphs. Many computational problems on graphs reduce to controlling or eliminating cycles — paths that return to their starting vertex. Cyclic structures create dependencies, feedback loops, and deadlocks in real systems, and their controlled removal is essential.

The **Feedback Vertex Set** (FVS) problem formalizes this cycle-elimination task:

> *Given a graph $G = (V, E)$, find a minimum-cardinality set $S \subseteq V$ such that the graph $G - S$ induced on $V \setminus S$ is acyclic.*

If G is undirected, acyclicity means a forest. If G is directed, acyclicity means a directed acyclic graph (DAG). The directed variant, DFVS, is similarly defined.

The problem arises concretely in:

- **Operating system deadlocks**: A resource-allocation graph where processes and resources form cycles must have critical processes removed to resolve deadlock.
- **Dependency resolution**: Software packages with circular dependencies require one dependency to be removed to restore a valid install order.
- **Circuit stabilization**: Feedback loops in electronic circuits, if uncontrolled, cause oscillations; removing cycle-forming components stabilizes the circuit.
- **Bayesian network structure learning**: Ensuring a learned graphical model is a DAG requires breaking directed cycles in candidate structures.

Despite its practical importance, FVS is computationally challenging. The decision version — does a feedback vertex set of size at most k exist? — is NP-Complete [1], and the optimization version is NP-Hard. This complexity gap motivates a multi-strategy approach: exact algorithms for small instances, heuristic algorithms for large instances, and hybrid methods that combine the strengths of both.

1.2 Problem Statement

Definition 1.1 (Feedback Vertex Set)**.** Let $G = (V, E)$ be an undirected graph. A **Feedback Vertex Set** of G is a subset $S \subseteq V$ such that $G[V \setminus S]$ is acyclic (a forest). A **minimum FVS** minimizes $|S|$.

Definition 1.2 (Directed Feedback Vertex Set)**.** Let $G = (V, E)$ be a directed graph. A **Directed Feedback Vertex Set** of G is a subset $S \subseteq V$ such that $G[V \setminus S]$ is a directed acyclic graph (DAG). A **minimum DFVS** minimizes $|S|$.

Definition 1.3 (Decision and Optimization Versions)**.** The **decision version** of FVS is: given $G = (V, E)$ and a non-negative integer k, does there exist $S \subseteq V$ with $|S| \leq k$ such that $G - S$ is acyclic? The **optimization version** seeks a minimum-size S.

1.3 Objectives of This Project

This project pursues the following objectives:

1. Implement exact FPT solvers (Iterative Compression and Bounded Search Tree) for both undirected and directed FVS.
2. Design, implement, and evaluate the KMA algorithm, integrating graph-theoretic kernelization with a population-based memetic search.
3. Design and evaluate DKMA, a dynamic extension of KMA that interleaves search and re-kernelization.
4. Design and evaluate three generations of GNN-guided hybrid solvers (GNN-KMA v1/v2/v3 and GNN-DKMA) that use learned vertex-level predictions to guide combinatorial search.
5. Construct and release the largest labeled FVS dataset publicly available, comprising 200K graphs across diverse families.
6. Benchmark all solvers on synthetic datasets and the PACE 2022 competition instances, with comparison against the official PACE winner.

1.4 Summary of Contributions

This report makes the following contributions:

1. **KMA:** First-class integration of kernelization (including directed SCC reduction) into a C++ memetic algorithm, eliminating 60–90% of directed-graph vertices before search begins.
2. **DKMA:** A dynamic re-kernelization loop with SHORTONE contractions, topological diversification, and 1-1/2-1 gain local search, inspired by PACE 2022 winning strategies.
3. **GNN-KMA v1/v2/v3:** Three generations of GNN architectures (GraphSAGE $\rightarrow$ GATv2, 3 $\rightarrow$ 16 feature channels, NLL $\rightarrow$ asymmetric precision loss), coupled to KMA under a precision-first strategy that caps the hard-fix ratio at 8%.
4. **GNN-DKMA:** Combination of GNN pre-fixing with DKMA's dynamic kernelization.
5. **FVS Dataset:** 200K labeled graphs (100K undirected, 100K directed) spanning five graph families, two difficulty tracks, with ground-truth solver labels, publicly released on Kaggle.
6. **Comprehensive Evaluation:** Benchmarking on synthetic data and PACE 2022, achieving 78.93/100 on 200 competition instances.

1.5 Report Organization

Chapter 2 provides theoretical prerequisites including graph theory, complexity theory, and the FPT framework. Chapter 3 reviews related work. Chapters 4–8 present the algorithms in detail. Chapter 9 covers implementation architecture. Chapter 10 presents datasets and experimental evaluation. Chapter 12 discusses insights and limitations. Chapter 14 concludes.

Chapter 2

Theoretical Background

2.1 Graph-Theoretic Foundations

Definition 2.1 (Graph)**.** A **graph** is a pair $G = (V, E)$ where V is a finite vertex set and E is an edge set. For undirected graphs, $E \subseteq \binom{V}{2}$; for directed graphs, $E \subseteq V \times V$.

Definition 2.2 (Cycle, Tree, and Forest)**.** An undirected **cycle** is a closed path $v_1, v_2, \ldots, v_\ell, v_1$ with $\ell \geq 3$ and distinct internal vertices. A graph without cycles is **acyclic**. A connected acyclic graph is a **tree**, and a disjoint union of trees is a **forest**.

Definition 2.3 (Feedback Vertex Set)**.** Given an undirected graph $G = (V, E)$, a set $S \subseteq V$ is a **Feedback Vertex Set** if $G[V - S]$ is acyclic. The optimization goal is to minimize $|S|$.

Definition 2.4 (Directed Feedback Vertex Set)**.** Given a directed graph $G = (V, E)$, a set $S \subseteq V$ is a **Directed Feedback Vertex Set** if $G[V - S]$ is a directed acyclic graph (DAG).

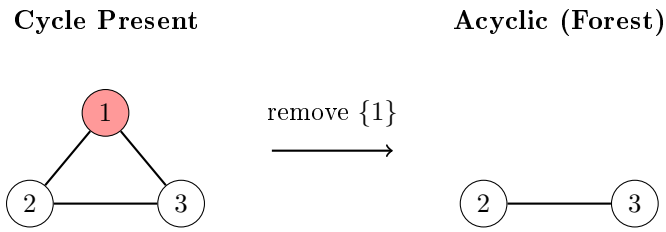


Figure 2.1: Basic FVS intuition: removing a vertex from every cycle makes the graph acyclic.

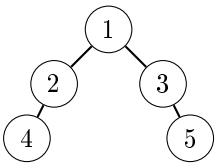
Intuition. The FVS objective can be interpreted as a cycle-hitting problem: every cycle must contain at least one removed vertex. The minimum FVS is the smallest set that intersects all cycles.

2.1.1 Core Observations

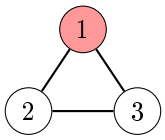
Three observations from the visual examples are fundamental for algorithm design:

1. Every cycle must contain at least one selected (removed) vertex.
2. The objective is combinatorial minimization: fewer removed vertices is always better.
3. Denser and more overlapping cycle structures typically increase search difficulty.

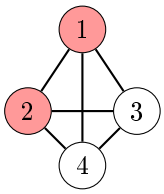
2.1.2 Additional Visual Examples



(a) A tree (already acyclic): FVS = $\emptyset$.

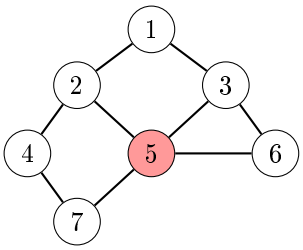


(b) Single triangle: removing one vertex breaks all cycles.

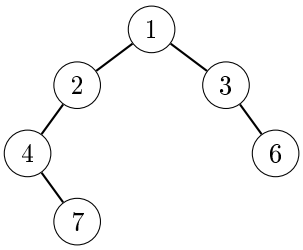


(c) Overlapping cycles may require multiple removals.

Figure 2.2: From simple to harder structures: cycle interaction determines FVS size.



(a) Original complex graph. Vertex 5 is the selected feedback vertex.



(b) Graph after removing $\{5\}$: two tree components, hence a forest.

Figure 2.3: Complex graph example where removing only node 5 makes the graph acyclic.

In the left graph of Figure 2.3, every cycle goes through vertex 5 (for example, $1 \to 2 \to 5 \to 3 \to 1$, $3 \to 5 \to 6 \to 3$ and $2 \to 4 \to 7 \to 5 \to 2$ in undirected notation). Therefore, removing node 5 alone destroys all cycles. The resulting graph on the right is a forest, so this example demonstrates a single-vertex solution with $S = \{5\}$.

2.2 Problem Formulations

Definition 2.5 (Decision Version)**.** Input: a graph $G = (V, E)$ and an integer $k \geq 0$. Question: does there exist a set $S \subseteq V$ with $|S| \leq k$ such that $G[V - S]$ is acyclic?

Definition 2.6 (Optimization Version)**.** Find a minimum-cardinality feedback vertex set.

The decision and optimization versions are tightly related: binary search on k with a decision oracle solves the optimization version. In complexity terms, the decision version is NP-Complete, and the optimization version is NP-Hard.

2.3 Complexity-Theoretic Background

Definition 2.7 (Class P)**.** Class **P** contains decision problems solvable in polynomial time.

Definition 2.8 (Class NP)**.** Class **NP** contains decision problems for which a proposed solution can be verified in polynomial time.

Definition 2.9 (NP-Complete)**.** A decision problem is **NP-Complete** if it is in NP and every problem in NP can be polynomially reduced to it.

2.3.1 Why FVS Is in NP

Given a candidate set S, we can verify in polynomial time whether $|S| \leq k$ and whether $G[V - S]$ is acyclic using DFS/BFS. This takes $\mathcal{O}(|V| + |E|)$ time. Hence, FVS belongs to NP.

2.3.2 NP-Hardness via Reduction from Vertex Cover

To prove NP-Completeness, we reduce from Vertex Cover, which is NP-Complete.

Theorem 2.1. The decision version of FVS is NP-Complete.

Proof sketch. Membership in NP was shown above. For NP-hardness, let (G, k) be a Vertex Cover instance with $G = (V, E)$. Construct a graph $G' = (V', E')$ as follows:

- Start with all original vertices and edges of G.
- For each edge $e = (u, v) \in E$, add a fresh vertex w_e and edges (u, w_e) and (v, w_e).

Each original edge now belongs to a triangle (u, v, w_e). Keep $k' = k$.

If $C \subseteq V$ is a vertex cover of size at most k, then every edge (u, v) has at least one endpoint in C, so every triangle (u, v, w_e) is hit by C; thus C is an FVS of G'. Conversely, let S' be an FVS of G' with $|S'| \leq k$. Any selected gadget vertex w_e can be replaced by either u or v without increasing size; therefore there exists an equal-size FVS $S \subseteq V$. Since triangle (u, v, w_e) must be hit by S and $w_e \notin S$, at least one of u, v lies in S. Hence, S is a vertex cover of G of size at most k.

Therefore, (G, k) is a YES-instance of Vertex Cover iff (G', k) is a YES-instance of FVS. The reduction is polynomial, so FVS is NP-Hard. Combining both parts, FVS is NP-Complete [1]. □

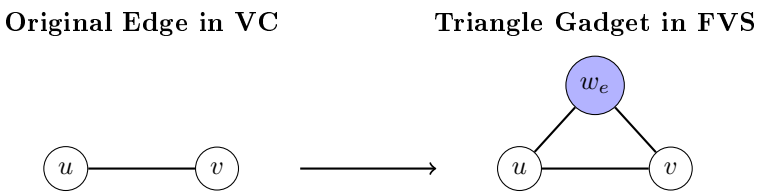


Figure 2.4: Edge-to-triangle gadget used in the reduction Vertex Cover $\leq_p$ FVS.

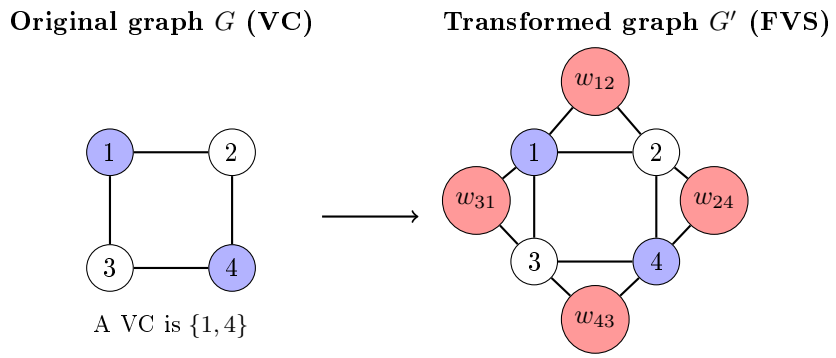


Figure 2.5: Full reduction illustration: each original edge is converted to a triangle gadget. Breaking all triangles corresponds to covering all original edges.

Correctness intuition for the reduction. The forward direction is immediate: a vertex cover intersects every edge, hence intersects every triangle in G'. For the backward direction, an FVS must hit every triangle, so at least one endpoint of each original edge must be selected (after replacing gadget vertices

by endpoints when needed). Therefore, selecting a minimum FVS in G' captures the same combinatorial constraint as selecting a minimum vertex cover in G.

2.3.3 Reverse Reduction Insight (FVS → VC)

A reverse construction idea showing close structural relation between the problems: replace each original vertex v by a triangle $\{v_1, v_2, v_3\}$, and encode original edges via cross-links between corresponding gadgets. Intuitively, selecting all three vertices of a triangle corresponds to placing v in FVS, while selecting only two corresponds to keeping v. This yields the parameter relation $k' = 2|V| + k$.

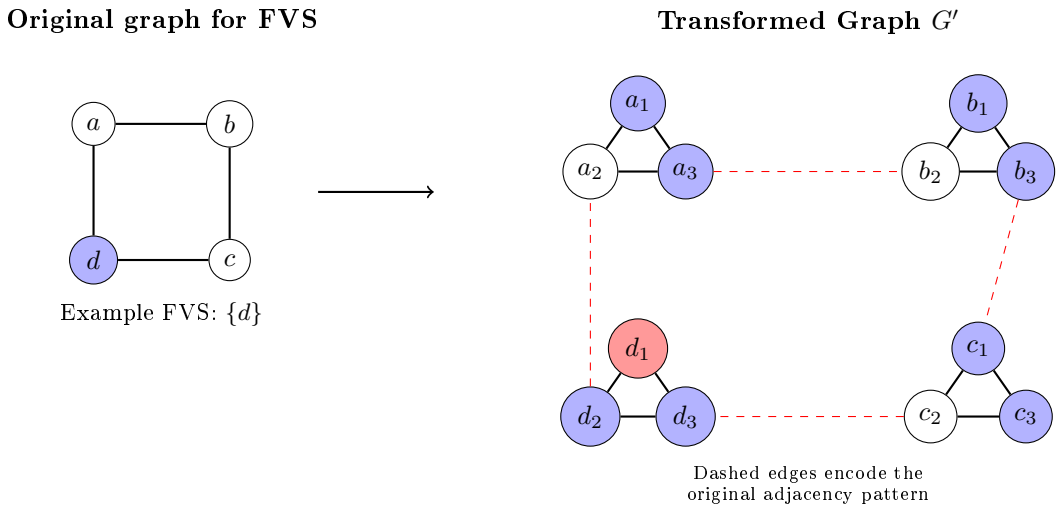


Figure 2.6: Reverse Reduction Insight: FVS to VC gadget construction with triangle replacements and cross-edges.

2.4 Directed Graph Perspective (SCC View)

Definition 2.10 (Strongly Connected Component (SCC))**.** In a directed graph, an SCC is a maximal vertex set C such that for every $u, v \in C$, there are directed paths $u \rightsquigarrow v$ and $v \rightsquigarrow u$.

Proposition 2.2. Every directed cycle is fully contained in a nontrivial SCC.

Hence, vertices outside nontrivial SCCs cannot participate in directed cycles and are irrelevant for DFVS. This observation is the basis of several practical reductions.

Chapter 3

Related Work

3.1 Overview

The Feedback Vertex Set (FVS) problem has been studied extensively due to its theoretical importance and wide range of practical applications. Since the problem is NP-Complete, no known polynomial-time algorithm exists for solving it optimally in general graphs. As a result, researchers have explored multiple classes of algorithms, each making different trade-offs between optimality, efficiency, and scalability.

Broadly, existing approaches can be categorized into:

- Exact exponential and parameterized algorithms
- Approximation algorithms
- Randomized algorithms
- Heuristic methods
- Metaheuristic techniques

Each category is discussed in detail below.

3.2 Exact Exponential and Parameterized Algorithms

3.2.1 Motivation

Exact algorithms aim to compute the *optimal* solution, i.e., the smallest possible feedback vertex set. These methods are theoretically important, but their computational cost grows exponentially with input size.

They are most useful when:

- Graph size is small, or
- Solution size k is small (FPT setting)

3.2.2 Naive Brute Force

The simplest approach is to try all possible subsets of vertices.

- For each subset $S \subseteq V$, remove S

- Check whether $G - S$ is acyclic
- Return the smallest valid set

Insight: This guarantees optimality but is computationally infeasible.

Complexity: $\mathcal{O}(2^n \cdot n^3)$

3.2.3 Dynamic Programming on Subsets

Dynamic programming improves brute force by avoiding redundant computations.

- Store results for previously evaluated subsets
- Reuse computations for overlapping subproblems

Complexity: $\mathcal{O}(2^n \cdot n^2)$

Observation: Still exponential, but more efficient than naive enumeration.

3.2.4 Iterative Compression

Iterative Compression is one of the most important techniques for FVS.

Core Idea:

- Start with a small graph and known solution
- Add vertices one by one
- Maintain a solution of size $k + 1$
- Compress it to size k

Why it works: The compression step transforms the problem into a structured subproblem.

Complexity: $\mathcal{O}(5^k \cdot kn^2)$

Key Strength: Highly effective for small parameter k.

3.2.5 Bounded Search Tree

This method explores possible solutions using branching.

- Find a cycle in the graph
- Select a vertex from the cycle
- Recursively branch on removing each vertex

Key Insight: Every cycle must contain at least one vertex in the FVS.

Complexity: $\mathcal{O}(4^k \cdot n^2)$

3.2.6 Advanced FPT Algorithms

Modern algorithms combine:

- Kernelization
- Sophisticated branching strategies

- Structural graph reductions

Best known complexity: $\mathcal{O}\left(3.619^k \cdot n^4\right)$

Observation: These methods are state-of-the-art for exact solutions.

3.3 Approximation Algorithms

3.3.1 Motivation

Approximation algorithms sacrifice optimality in exchange for efficiency. They guarantee solutions close to optimal within a provable bound.

3.3.2 2-Approximation Algorithm

Idea:

- Find a cycle
- Pick an edge from the cycle
- Remove both endpoints

Why it works: At least one endpoint must be in the optimal solution.

Guarantee: $|S| \leq 2 \cdot OPT$

3.3.3 Logarithmic Approximation

This method reduces FVS to a hitting set problem.

- Each cycle is treated as a set
- Select vertices that hit all sets

Guarantee: $\mathcal{O}(\log n)$

3.3.4 LP Relaxation and Rounding

- Formulate as Integer Linear Program (ILP)
- Relax to Linear Program (LP)
- Round fractional values to integers

Complexity: $\mathcal{O}\left(n^{3.5}\right)$

Advantage: Provides systematic approximation.

Limitation: High computational cost.

3.4 Randomized Algorithms

3.4.1 Motivation

Randomized algorithms use probabilistic decisions to improve performance.

Key Idea: Instead of deterministic choices, use randomness to explore solution space.

3.4.2 Random Sampling

- Randomly select vertices
- Evaluate whether they break cycles
- Repeat multiple times

Benefit: Fast in practice.

Expected Performance: $\mathcal{O}(\log n)$ approximation

3.4.3 Randomized Rounding

- Use LP solution
- Interpret fractional values as probabilities
- Randomly choose vertices

Expected Guarantee: 2-approximation

3.4.4 Color Coding

Idea:

- Assign random colors to vertices
- Identify specific substructures

Complexity: $\mathcal{O}\big(2^{O(k)} \cdot n\big)$

3.5 Heuristic Algorithms

3.5.1 Motivation

Heuristic algorithms aim for fast and practical solutions without guarantees.

They are widely used in real-world applications where:

- Graphs are large
- Exact solutions are infeasible

3.5.2 Greedy Vertex Selection

- Select highest-degree vertex
- Remove it
- Repeat until no cycles remain

Insight: High-degree vertices are likely part of many cycles.

Complexity: $\mathcal{O}\big(n^2 \log n\big)$

3.5.3 Cycle-Based Greedy

- Detect cycles
- Count vertex frequency
- Remove most frequent vertex

Advantage: More cycle-aware than degree-based methods.

Complexity: $\mathcal{O}(n^3)$

3.5.4 Core-Based Methods

- Compute k-core decomposition
- Focus on dense regions

Insight: Cycles are more likely in dense subgraphs.

Complexity: $\mathcal{O}(n + m)$

3.5.5 Reduction Rules + Greedy

- Remove degree ≤ 1 vertices
- Apply kernelization rules
- Run greedy on reduced graph

Benefit: Significantly reduces problem size.

Complexity: $\mathcal{O}(n^2)$

3.6 Metaheuristic Algorithms

3.6.1 Motivation

Metaheuristics are global optimization techniques designed to explore large solution spaces.

They are particularly effective for:

- Large-scale graphs
- Complex optimization landscapes

3.6.2 Genetic Algorithms

- Maintain population of solutions
- Apply crossover and mutation
- Select best individuals

Strength: Explores diverse solutions.

Complexity: $\mathcal{O}(g \cdot p \cdot n^2)$

3.6.3 Simulated Annealing

- Accept worse solutions probabilistically
- Gradually reduce randomness

Advantage: Escapes local minima.

Complexity: $\mathcal{O}(i \cdot n^2)$

3.6.4 Ant Colony Optimization

- Simulate ants constructing solutions
- Use pheromone trails to guide search

Strength: Collective learning behavior.

Complexity: $\mathcal{O}(\text{ants} \cdot i \cdot n^2)$

3.6.5 Tabu Search

- Explore neighborhood solutions
- Maintain tabu list to avoid repetition

Complexity: $\mathcal{O}(i \cdot n^2)$

3.6.6 Particle Swarm Optimization

- Represent solutions as particles
- Update based on global and local best

Complexity: $\mathcal{O}(\text{particles} \cdot i \cdot n^2)$

3.7 Summary and Positioning of Our Work

From the above survey, we observe a clear trade-off:

- Exact methods: optimal but slow
- Heuristic methods: fast but approximate
- Metaheuristics: scalable but not guaranteed

Our work builds upon these approaches by combining:

- Kernelization (from parameterized algorithms)
- Evolutionary search (from metaheuristics)
- Learning-based guidance (Graph Neural Networks)

This hybrid framework aims to achieve both scalability and high-quality solutions, bridging the gap between theoretical optimality and practical performance.

Chapter 4

Exact Algorithms

4.1 Iterative Compression

4.1.1 Core Idea

Iterative Compression (IC) is an FPT algorithm for FVS that builds a solution incrementally, compressing an $(k+1)$-sized FVS to a k-sized FVS at each step.

Algorithm 1 Iterative Compression for FVS

Require: Graph $G = (V, E)$, parameter k

Ensure: Minimum FVS of size $\leq k$, or NO

```
Order vertices v_1, v_2, ..., v_n
F ← {v_1}
for i = 2 to n do
    F ← F ∪ {v_i}
    if |F| = k + 1 then
        Enumerate all partitions (F_1, F_2) of F
        for each partition (F_1, F_2) do
            Find Y ⊆ V \ F_2 such that F_1 ∪ Y is an FVS of G[V \ F_2] of size ≤ k − |F_1|
            if such Y exists then
                F ← F_1 ∪ Y; break
            end if
        end for
        if no valid partition found then return NO
        end if
    end if
end forreturn F if |F| ≤ k
```

4.1.2 Complexity Analysis

At each compression step, there are 2^{k+1} partitions of F. For each partition, the inner compression problem on the disjoint vertex set can be solved in $\mathcal{O}\big(4^{k-|F_1|} \cdot n^2\big)$ using a disjoint-paths-based subroutine. Over n steps, the total complexity is $\mathcal{O}\big(5^k \cdot k \cdot n^2\big)$ [5].

The IC solver is implemented in C++ in the `cpp_engine` module. It is exact, guaranteed to find the

optimal solution or correctly report that no solution of size $\leq k$ exists. In our evaluation, IC completes 9,906 exact-track instances with 100% optimality before timeout.

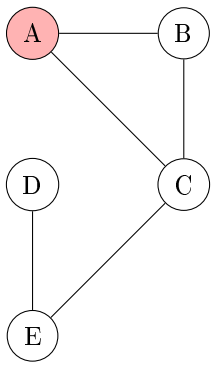
4.1.3 Advantages and Limitations

IC provides a guaranteed optimal solution and is theoretically well-grounded. Its limitation is the exponential dependence on k: for $k > 30$, it becomes infeasible within practical time budgets. On the exact track (graph sizes 10–35 nodes), it performs well.

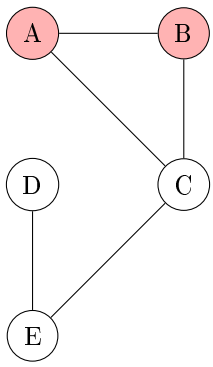
4.1.4 Simulation

Simulation of FVS (k=2) on 5 vertices

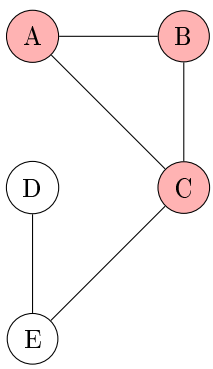
Step 1: $F = \{A\}$



Step 2: $F = \{A, B\}$

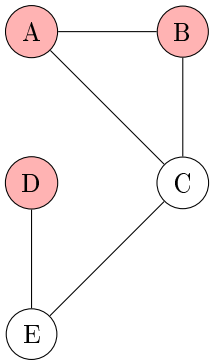


Step 3: $F = \{A, B, C\}$ (overflow)



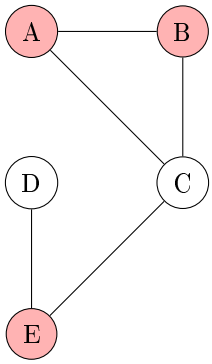
Fix: Remove C $\Rightarrow F = \{A, B\}$

Step 4: Add D (Overflow)



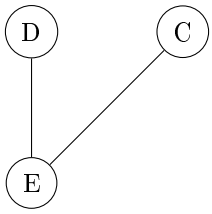
Fix: Remove D $\Rightarrow F = \{A, B\}$

Step 5: Add E (Overflow)



Fix: Remove E $\Rightarrow F = \{A, B\}$

Final Answer



$F = \{A, B\}$

4.2 Bounded Search Tree (BST)

4.2.1 Algorithm Description

The Bounded Search Tree algorithm applies kernelization as preprocessing, reducing the graph to a smaller equivalent kernel, then exhaustively branches over remaining cycles.

Algorithm 2 Kernelization + Bounded Search Tree

Require: Graph G, parameter k

Ensure: Minimum FVS, or NO

```
Apply reduction rules:
    Remove vertices of degree ≤ 1 (cannot be in any cycle)
    Contract degree-2 vertices via bypass rule
    Force self-loop vertices into FVS and remove
Obtain kernel (G', k')
function SEARCH(G', k')
    if G' is acyclic then return ∅
    end if
    if k' < 0 then return NO
    end if
    Find a cycle C in G'
    best ← ∞
    for each v ∈ C do
        sol ← v ∪ SEARCH(G' − v, k' − 1)
        best ← min(best, sol)
    end forreturn best
end function
```

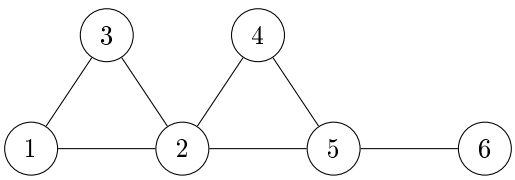
4.2.2 Complexity Analysis

After kernelization, the kernel has $\mathcal{O}(k^2)$ vertices [3]. The search tree branches on a cycle (branching factor $\leq |C|$), with depth k. The total complexity is $\mathcal{O}(4^k + n^2)$ in the combined kernel + search-tree analysis.

BST completes 9,046 exact-track instances with 100% optimality before timeout in our evaluation.

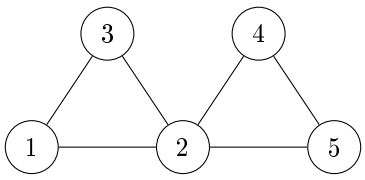
4.2.3 Simulation

Step 1: Original Graph

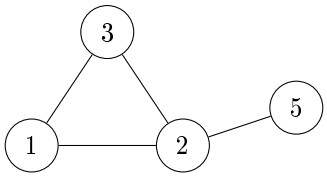


Step 2: Remove degree ≤ 1 vertices

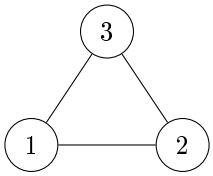
Vertex 6 removed



Step 3: Contract degree-2 vertex (4)

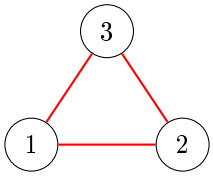


Step 4: Remove degree ≤ 1 vertex (5)

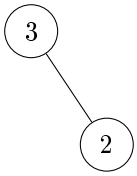


Step 5: Kernel Graph (G')

Remaining cycle

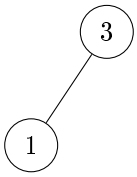


Step 6: Branch - Remove vertex 1



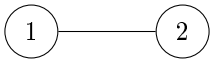
Graph becomes acyclic

Step 7: Branch - Remove vertex 2



Graph becomes acyclic

Step 8: Branch - Remove vertex 3



Graph becomes acyclic

Chapter 5

The Memetic Algorithm

5.1 Algorithm Overview

The Memetic Algorithm (MA) is an evolutionary heuristic that combines genetic-algorithm population dynamics with problem-specific local search. It is designed for large instances ($n > 1000$) where exact methods are infeasible.

Algorithm 3 Memetic Algorithm for FVS

Require: Graph G, population size P, max generations $G_{\max}$, early-stop T
Ensure: Near-optimal FVS

```
Encode solutions as vertex permutations
Initialize population: P/2 greedy, P/2 random individuals
for g = 1 to G_max do
    Evaluate fitness: −|FVS(ind)| for each individual ind
    Select parents via tournament selection
    Produce offspring via cycle-aware crossover
    Apply mutation (swap cycle-heavy vertices)
    Apply local search (hill-climbing on each offspring)
    Repair: restore FVS validity via greedy cycle cover
    Update population via tournament replacement
    if no improvement for T generations then break
    end if
end forreturn best individual decoded as FVS
```

5.2 Encoding and Decoding

Each individual encodes a **permutation** of vertices $\pi = (\pi_1, \pi_2, \ldots, \pi_n)$. Decoding proceeds greedily: vertices are removed in order $\pi_1, \pi_2, \ldots$ until the remaining graph is acyclic. This ensures every individual represents a valid (though not necessarily minimum) FVS.

5.3 Genetic Operators

Crossover. Given two parent permutations, a cycle-aware crossover operator produces offspring by preserving cycle-heavy vertices from the better parent and inheriting the relative order of remaining

vertices from the other.

Mutation. A vertex swap operator selects a vertex with high cycle centrality (part of many cycles) and swaps its position earlier in the permutation, increasing its probability of inclusion in the decoded FVS.

Local Search. For each individual, a hill-climbing post-process attempts to reduce FVS size: it tries removing each FVS vertex and checks whether acyclicity is preserved; if so, the vertex is kept out. This guarantees a local optimum with respect to single-vertex removals.

5.4 Complexity and Parameters

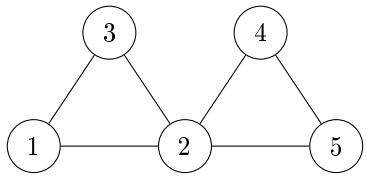
The MA runs in $\mathcal{O}\big(P \cdot G_{\max} \cdot |V|^2\big)$ per generation due to cycle detection. Default parameters are given in Table 5.1.

Table 5.1: Default MA / KMA parameters

Parameter	Flag	Default	Description
Population	`-pop`	20	Individuals per generation
Generations	`-gens`	100	Max evolutionary iterations
Early-stop	`-earlystop`	30	Gens without improvement
Timeout	`-timeout`	600s	Hard wall-clock limit

5.4.1 Simulation

Step 1: Original Graph

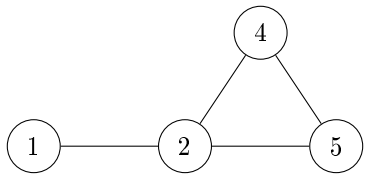


Step 2: Encode Solution

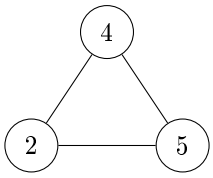
Example permutation:

$$[3,\ 1,\ 5,\ 2,\ 4]$$

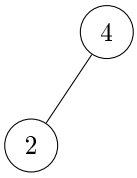
Step 3: Remove vertex 3



Step 4: Remove vertex 1



Step 5: Remove vertex 5



Graph is now acyclic → Stop decoding

Step 6: Fitness Evaluation

Removed vertices = {3,1,5}

$$\text{Fitness} = -|FVS| = -3$$

Step 7: Selection (Tournament)

Select best solutions from population

Example:

$$[3, 1, 5, 2, 4], \quad [2, 4, 1, 5, 3]$$

Step 8: Crossover

Parent 1: [3,1,5,2,4]

Parent 2: [2,4,1,5,3]

Child:

$$[3,4,1,2,5]$$

Step 9: Mutation

Before:

$$[3, 4, 1, 2, 5]$$

After swap:

$$[3,1,4,2,5]$$

Step 10: Local Search

Try improving solution:

Remove fewer vertices if possible

Example improvement:

$$\text{FVS size: } 3 \rightarrow 2$$

Step 11: Best Solution

Best FVS found:

$$\{3,2\}$$

$$\text{Graph becomes acyclic}$$

Chapter 6

KMA: Kernelized Memetic Algorithm

6.1 Motivation

The plain Memetic Algorithm operates on the full input graph, wasting effort on vertices that can be deterministically resolved by reduction rules. KMA applies graph-theoretic kernelization *before* the MA, reducing the search space and focusing the evolutionary search on structurally hard subgraphs.

6.2 KMA Pipeline

The KMA pipeline has three phases:

1. **Kernelization**: Apply safe reduction rules, collecting forced vertices.
2. **MA on kernel**: Run the C++ MA solver on the reduced kernel G'.
3. **Reconstruction**: Map kernel-vertex indices back to original indices and union with forced vertices.

Algorithm 4 KMA: Kernelized Memetic Algorithm

Require: Graph $G = (V, E)$, MA parameters
Ensure: Near-optimal FVS

```
(G', k', F_forced, φ) ← KERNELIZE(G)                ▷ φ: kernel-to-original map
if |V(G')| = 0 then return F_forced
end if
S' ← MA(G', parameters)                             ▷ C++ solver on kernel
S ← F_forced ∪ {φ(v) : v ∈ S'} return deduplicate(S)
```

6.3 Kernelization Rules

6.3.1 Undirected Kernelization

The function `kernelize_undirected_graph` applies the following rules iteratively to quiescence:

1. **Self-loop rule**: If $(v, v) \in E$, force v into FVS and remove.
2. **Degree-0/1 rule**: If $\deg(v) \leq 1$, remove v (cannot be in any cycle).

3. **Degree-2 bypass rule**: If $\deg(v) = 2$ with neighbors a, b and $\{a, b\} \notin E$, replace v and its edges with a single edge (a, b).

These rules are safe: no optimal FVS excludes forced vertices, and removed vertices never need to be included. The theoretical basis is the $\mathcal{O}(k^2)$ kernel of Thomassé [3].

6.3.2 Directed Kernelization

The function `kernelize_directed_graph` applies stronger directed rules:

1. **Self-loop rule**: If $(v, v) \in E$, force v into DFVS and remove.
2. **Source/Sink rule**: If in-deg$(v) = 0$ or out-deg$(v) = 0$, remove v (not on any directed cycle).
3. **Degree-1 bypass**: If in-deg$(v) = 1$ or out-deg$(v) = 1$, shortcut its predecessor(s) to its successor(s) and remove.
4. **SCC reduction**: Compute SCCs via Tarjan's algorithm [16]; remove all vertices not in nontrivial SCCs. A nontrivial SCC has $|C| > 1$ or contains a self-loop.

Key Impact of SCC Reduction

SCC-based directed reduction alone eliminates 60–90% of vertices on sparse real-world directed graphs before any search begins, dramatically shrinking the kernel and making DFVS tractable at scales impossible for undirected FVS with comparable vertex counts.

6.4 C++ Backend Solver

The MA on the kernel is dispatched to the C++ engine (`cpp_engine`), which exposes:

- `cpp_engine.solve_undirected_KMA` / `solve_directed_KMA`
- Fallback chain: KMA $\rightarrow$ KME $\rightarrow$ MA

The C++ solver enforces a hard wall-clock timeout internally, returning the best solution found within the budget.

Chapter 7

DKMA: Dynamic Kernelized Memetic Algorithm

7.1 Motivation: The Opening-Up Effect

KMA applies kernelization once before search. A fundamental limitation is that partial solutions formed during evolutionary search may enable additional reductions that static pre-kernelization cannot exploit. Consider a directed graph where, after one generation of MA, 80% of individuals agree that vertex v belongs to the FVS. If v is committed, its removal may create new sources/sinks or break SCCs, unlocking further reductions. This **opening-up effect** [8] progressively shrinks the kernel as partial consensus is reached.

7.2 DKMA Algorithm

Algorithm 5 DKMA: Dynamic Kernelized Memetic Algorithm

Require: Graph G, threshold τ, MA parameters
Ensure: Near-optimal FVS

```
(G', F, φ) ← StaticKernelize(G)
if directed then Apply ShortonePas(G')
end if
P ← InitPopulation(G')
repeat
    P ← RunOneGeneration(P, G')
    C ← CommitVertices(P, G', τ)                ▷ Vertices in ≥ τ fraction
    F ← F ∪ φ(C); remove C from G'
    (G', φ') ← DynamicKernelize(G')             ▷ Re-kernelize residual
    φ ← φ ∘ φ'
    if every 5 generations then P ← TopologicalDiversify(P, G')
    end if
until convergence or timeout
S ← GainLocalSearch(F ∪ φ(best(P)), G)          ▷ 1-1 and 2-1 swaps
Verify acyclicity; fallback to KMA if invalid return S
```

7.3 Key Innovations

7.3.1 Commit Threshold and Cap

The function `_commit_vertices_from_population` selects consensus vertices:

- A vertex v is committed if it appears in $\geq \tau$ fraction of population individuals (default $\tau = 0.8$).
- If too many vertices would be committed, cap at the top 30% by frequency to prevent over-committing and loss of diversity.

This is based on the DFVS approach from Grossmann et al. [8].

7.3.2 SHORTONE Directed Contractions

Inspired by the Mount-Doom PACE 2022 solver [10], the function `_apply_shortone_rule` performs a directed contraction pass before the MA loop. It contracts degree-1 in/out vertices with cycle-preserving rewiring, enabling additional SCC reductions that standard kernelization misses.

7.3.3 Topological Diversification

Every 5 generations, `_diversify_with_topological_ordering` is called to prevent premature convergence [7]:

- **Directed**: Randomized Kahn's algorithm with tie-breaking on cycle-heavy vertices.
- **Undirected**: Random spanning-tree with cycle-breaking insertion.

7.3.4 Gain Local Search

After the main loop, `_gain_local_search` applies improvement swaps:

- **1-1 swap**: Remove one FVS vertex and add another; accept if FVS remains valid and size decreases.
- **2-1 swap**: Replace two FVS vertices with one; accept if valid.

7.3.5 Correctness Safeguards

DKMA enforces:

1. Deduplicated final solution indices.
2. All forced vertices (static + dynamic) are included.
3. Final acyclicity verification on the *original* graph: directed uses iterative DFS back-edge detection; undirected uses union-find.
4. Fallback to KMA if final verification fails.

7.4 KMA vs. DKMA: Comparison

Table 7.1: Feature comparison between KMA and DKMA

Feature	**KMA**	**DKMA**
Kernelization	Once (static)	Per-generation loop (dynamic)
Vertex commitment	None	Consensus-based ($\tau = 0.8$)
SHORTONE pass	No	Yes (directed only)
Topological diversification	No	Every 5 generations
Post-loop local search	No	Gain 1-1 / 2-1 swaps
Final validity check	End	End + KMA fallback

Chapter 8

GNN-KMA Models

8.1 Motivation: Why Learn to Guide Search?

Both KMA and DKMA apply reduction rules and evolutionary search but are agnostic to the learned patterns in graph structure that correlate with FVS membership. A graph neural network (GNN) trained on solved instances can learn which structural features make a vertex likely to be in the optimal FVS, providing *soft hints* that guide the solver toward better solutions.

8.2 Precision-First Coupling Strategy

The core coupling design is **precision-first soft-hint**:

1. Kernelize the input graph G to obtain kernel G'.
2. Run GNN inference on G' to obtain per-vertex probabilities $p(v)$.
3. Hard-fix only vertices with $p(v) \geq \theta$ (default $\theta = 0.65$).
4. Cap hard-fix ratio at 8% of kernel size.
5. Run KMA on the residual kernel $G'' = G' \setminus \{\text{hard-fixed}\}$.
6. Reconstruct: $S = F_{\text{forced}} \cup F_{\text{GNN}} \cup \phi(\text{KMA}(G''))$.

Why Precision-First?

False positives in hard-fixing are catastrophic: a wrongly fixed vertex inflates the final FVS size and cannot be removed within that solve branch. Capping at 8% of the kernel and requiring $\theta \geq 0.65$ ensures the GNN only commits when highly confident. If no vertex exceeds θ, the system automatically falls back to pure KMA.

8.3 GNN-KMA v1: Baseline Architecture

8.3.1 Feature Engineering (3 channels)

Undirected:

$$x_1(v) = \frac{\deg(v)}{\max(n-1,1)} \qquad \text{(normalized degree)} \tag{8.1}$$

$$x_2(v) = c(v) \qquad \text{(clustering coefficient)} \tag{8.2}$$

$$x_3(v) = \frac{\log(\deg(v)+1)}{\log(n+1)} \qquad \text{(normalized log-degree)} \tag{8.3}$$

Directed:

$$x_1(v) = \frac{d^-(v)}{\max(n-1,1)}, \quad x_2(v) = \frac{d^+(v)}{\max(n-1,1)}, \quad x_3(v) = \frac{\min(d^-(v), d^+(v))}{\max(n-1,1)} \tag{8.4}$$

The clustering coefficient [18] captures local cycle density, making it a natural predictor for FVS membership.

8.3.2 Model Architecture

UndirectedFVSNet: 3-layer GraphSAGE [11] message passing, BatchNorm + Dropout after each layer, MLP classification head, 2-class log-softmax output.

DirectedFVSNet: Custom `DirectedConvLayer` with separate in-aggregation, out-aggregation, and self terms; 3 layers + BN + Dropout + MLP; 2-class log-softmax output.

8.3.3 Training

Weighted NLL loss with class weights $w_c = n/(2 \cdot |C_c|)$, cosine annealing LR schedule, checkpoint by validation F1.

8.4 GNN-KMA v2: Richer Structural Features

v2 extends the feature set to 11 channels:

1. Degree-like channels (same as v1).
2. **RWSE** [13]: Random Walk Structural Encodings — short-range return probabilities at steps $\{2, 3, 4\}$.
3. **Motif counts** [14]: triangle count, 4-cycle participation, 4-clique-style counts.
4. **k-core number** $\kappa(v)$ [15]: vertices deep in dense cores are more likely to be in any FVS.

The model architecture retains the GraphSAGE backbone but with upgraded input (in_channels = 11). RWSE directly captures short-range cycle return probability, providing a continuous proxy for FVS membership likelihood.

8.5 GNN-KMA v3: Attention-Based Precision Model

v3 is designed for precision-first hybrid coupling, with every architectural and training choice oriented toward reducing false positives.

8.5.1 Feature Engineering (16 channels)

- Extended RWSE horizon: steps $\{2, 3, 4, 6, 8, 12, 16\}$.
- **SCC-aware channels** (directed): SCC size, SCC membership depth — vertices in large SCCs are more critical for DFVS.
- Cycle-score interaction features.
- Directional ratio features: $d^-/(d^- + d^+)$.
- Reduced reliance on expensive motif channels.

8.5.2 Model Architecture: DirectedFVSNetV3

1. Input projection layer.
2. 5 residual GATv2 blocks [12] with forward and reverse message paths.
3. Per-layer fusion projections.
4. Global readout context concatenated to each node embedding.
5. Single-logit output per node (sigmoid probability).

8.5.3 Training Innovations

- **AsymmetricFVSLoss**: Penalizes false positives more strongly than false negatives, directly matching the precision-first coupling requirement.
- **Warmup + cosine LR + gradient clipping**.
- **Checkpoint metric**: topk-precision@8% — the top 8% of predicted FVS vertices' precision, matching the hard-fix cap used at inference.
- 200 epochs (vs. 100 for v1/v2).

8.6 GNN-DKMA

GNN-DKMA applies the same GNN pre-fixing principle, but hands the residual kernel to DKMA instead of KMA, enabling dynamic re-kernelization atop GNN guidance.

8.7 Architecture Evolution Summary

Table 8.1: GNN-KMA v1/v2/v3 feature and architecture comparison

Aspect	**v1**	**v2**	**v3**
Feature channels	3	11	16
Clustering coeff.	✓	—	—
RWSE	—	steps $\{2, 3, 4\}$	steps $\{2 \ldots 16\}$
Motif counts	—	✓	reduced
k-core	—	✓	—
SCC channels	—	—	✓
Backbone	GraphSAGE	GraphSAGE	GATv2
Depth	3 layers	3 layers	5 residual blocks
Bidirectional	No	No	Yes (fwd+rev)
Global readout	No	No	Yes
Loss	NLL	NLL	AsymmetricFVSLoss
Checkpoint metric	Val F1	Val F1	topk-prec@8%
Epochs	100	100	200

Chapter 9

Implementation Architecture

9.1 System Architecture

The codebase is organized into four layers:

1. **C++ Algorithm Engine** (`cpp_engine/`): High-performance solver implementations (BST, IC, MA, KMA, DKMA, directed variants). Compiled as a Python extension module via `build_engine.py`.
2. **Python Orchestration** (`experiments/`): Benchmark drivers, pipeline runners, hybrid runtime, suite runner with resume semantics.
3. **Data Pipeline** (`data/`): Real-world graph download, synthetic graph generation, TXT-to-PT conversion with solver labels.
4. **GNN Stack** (`gnn_model/`): Feature engineering, model definitions, training loop, runtime inference integration.

9.2 Data Structures and Key Abstractions

Graph Representation. Graphs are stored as compact integer edge lists (0-indexed). Each `.txt` file follows the `edge_list_v1` format with a metadata header (`p edge N M`), directed/undirected flag, and source tag.

PT Dataset Format. Each PyG `Data` object stores: node feature matrix `x`, edge index `edge_index`, binary label vector `y`, `fvs_size`, and metadata (`family`, `track`, `category`, `source_file`, `feature_set`).

Kernel Representation. Kernelization returns `(k_n, k_edges, forced, k_new_to_old)`, where `k_new_to_old` maps kernel vertex indices back to original indices.

9.3 C++ Engine Dispatch

At runtime, the hybrid solver dispatches to the best available C++ solver:

```
for fn_name in ['solve_undirected_KMA', 'solve_undirected_KME', '
    solve_undirected_MA']:
    if hasattr(cpp_engine, fn_name):
        solver_fn = getattr(cpp_engine, fn_name)
```

```
        break
```

Listing 9.1: Solver dispatch in run_hybrid.py

9.4 Reproducibility and Resume Semantics

All benchmark pipelines support deterministic restart: each CSV output is appended one row at a time immediately after each file completes, and existing rows are skipped on resume. Synthetic generation uses deterministic per-bucket seed derivation from a global seed, ensuring identical datasets across runs with the same seed.

9.5 GNN Inference Integration

GNN loading is lazy: PyTorch, PyG, and model weights are imported only when needed, allowing the system to fall back gracefully to pure KMA if dependencies are unavailable. Feature computation at inference exactly mirrors dataset generation formulas to prevent train/inference mismatch.

Chapter 10

Datasets and Experimental Setup

10.1 Synthetic Dataset

10.1.1 Design Goals

The dataset generation pipeline is designed around five goals: (1) controlled distribution with fixed category proportions by family; (2) track-aware difficulty with separate exact and heuristic tracks; (3) reproducibility via deterministic seeding; (4) restartability via safe reuse of existing files; (5) benchmark/-train alignment ensuring GNN training data mirrors benchmark data distribution.

10.1.2 Dataset Composition

Table 10.1: Undirected graph categories and weights

Category	Weight	Graph Model
real_world	20%	NetworkX / PyG / OGB / SNAP
scale_free	20%	Barabási–Albert [17]
small_world	20%	Watts–Strogatz [18]
random_er	20%	Erdős–Rényi [19]
grids_trees	20%	Grid + Random tree

Table 10.2: Directed graph categories and weights

Category	Weight	Graph Model
real_world_ego	30%	SNAP ego-networks
scale_free	20%	Directed BA projection
random_er	20%	Directed Erdős–Rényi
directed_grids	15%	Oriented grid with reverse-edge injection
dags	15%	GN DAG-style generator

10.1.3 Track Size Intent

Table 10.3: Track-specific graph size ranges

Track	Node Range
exact_track	$\approx$ 10–35 nodes
heuristic_track	$\approx$ 100–5000 nodes

10.1.4 Ground-Truth Labels

Labels are obtained from exact solvers where feasible:

- **exact_track**: IC-based labels (high-fidelity, near-optimal, 60-second timeout).
- **heuristic_track**: KMA-based labels (practical large-instance behavior).

Each label is validated by an acyclicity check before saving. Invalid labels are discarded; a CSV status ledger tracks completed, timeout, and invalid outcomes.

10.1.5 Scale and Availability

The full dataset comprises **100K undirected + 100K directed** labeled FVS graphs, publicly released on Kaggle:

- https://www.kaggle.com/datasets/tawkirazizrahman/fvs-synthetic-dataset-100k
- https://www.kaggle.com/datasets/tawkirazizrahman/fvs-synthetic-dataset-20k

To our knowledge, this is the largest publicly available labeled FVS dataset.

10.2 PACE 2022 Benchmark

The Parameterized Algorithms and Computational Experiments (PACE) Challenge 2022 [8] focused on FVS and provided 200 public undirected test instances curated from real-world and synthetic sources, ranging from small exact instances to large heuristic-track graphs. The official solution checker and world-class solver implementations serve as baselines.

We use PACE 2022 as a held-out real-world stress test for heuristic solvers. The PACE winner (Sylwester Swat, Poznań University of Technology) achieved a reported score of 99.91/100.

10.3 Evaluation Metrics

- **FVS size**: Number of vertices in the computed solution (smaller is better).
- **Runtime**: Wall-clock seconds (includes kernelization and GNN inference).
- **Validity**: Boolean indicating whether the returned set is a valid FVS (acyclicity check).
- **PACE score**: $100 \times \frac{\mathrm{OPT_{FVS}}}{\text{solved FVS}}$, averaged over all instances.
- **Optimality**: For exact-track comparisons, percentage of instances matching the optimal solution.

10.4 Experimental Protocol

- Timeouts: $n \leq 50 \rightarrow 100$s; $50 < n \leq 200 \rightarrow 500$s; $n > 200 \rightarrow 600$s.
- Results logged to CSV with resume semantics.
- All benchmarks run on the same hardware for comparability.
- GNN inference uses the pre-trained weights from `gnn_model/weights/`.

Chapter 11

Results and Evaluation

11.1 Exact Solver Results

11.1.1 Optimality on Completed Instances

Table 11.1: Exact solver performance on undirected exact-track instances

Solver	Mean Score (%)	Instances Completed
Brute Force	100.0	7,263
BST Exact	100.0	9,046
IC Exact	100.0	9,906

All three solvers achieve 100% optimality on their completed instances, confirming the correctness of the C++ implementations. IC completes the most instances, consistent with its better theoretical complexity. All solvers produce identical solutions on shared instances, providing cross-validation of correctness.

11.1.2 Scalability Analysis

Exact success rates decline with graph size (n) and solution size (k). IC and BST maintain near-100% completion for $n \leq 20$ and $k \leq 10$, but success rates drop sharply beyond $n = 28$ and $k = 15$. This is expected from the exponential FPT complexity. The runtime vs. k curve (fixed $n = 28$) shows exponential growth for both solvers, while the runtime vs. n curve (fixed $k = 4$) shows polynomial growth, confirming the FPT behavior.

11.2 PACE 2022 Heuristic Results

Table 11.2: PACE 2022 benchmark results (200 instances, 600s total)

Solver	**Mean Score (/100)**	**Total Time (s)**
PACE 2022 Winner	100.00	600.0
directed KMA (pop=90)	78.93	18,709.9
directed GNN-KMA-3 θ=0.85 (pop=30)	78.64	15,617.5
directed GNN-KMA-3 θ=0.95 (pop=30)	77.75	15,302.6
directed GNN-KMA θ (pop=30)	76.98	8,380.7
directed DKMA (pop=30)	76.49	8,367.2
directed KMA (pop=30)	76.37	7,820.3
directed GNN-KMA-3 small θ=0.9 (pop=30)	76.27	16,480.6
directed DKMA (pop=15)	75.90	4,641.9
directed KMA (pop=15)	75.69	4,209.2
directed MA (pop=90)	71.84	18,281.5

11.3 Comparative Analysis

11.3.1 Kernelization Impact: MA vs. KMA

KMA (pop=30) achieves a mean PACE score of 76.37, compared to MA (pop=90) at 71.84. This 4.5-point improvement, despite MA using 3× the population, demonstrates the strong benefit of pre-search kernelization: eliminating vertices that kernelization rules can handle deterministically frees evolutionary budget for genuinely hard substructures.

11.3.2 Dynamic Kernelization: DKMA vs. KMA

DKMA (pop=30) achieves 76.49 vs. KMA (pop=30) at 76.37, a modest but consistent improvement. The opening-up effect is more pronounced on sparse graphs with hierarchical SCC structure, where dynamic commitment progressively unlocks reductions.

11.3.3 GNN Guidance: GNN-KMA vs. KMA

GNN-KMA v3 ($\theta = 0.85$, pop=30) achieves 78.64, **2.27 points above** baseline KMA at the same population, demonstrating that learned vertex-level predictions provide meaningful guidance. v3 outperforms v1 by approximately 1.6 points, reflecting the benefit of deeper features (16 channels, SCC-aware, extended RWSE) and the precision-oriented training objective.

11.3.4 Population Scaling

Increasing population from 30 to 90 boosts KMA from 76.37 to 78.93 at a 3× computational cost. This suggests that GNN-KMA v3 provides comparable benefit to 3× more evolutionary effort, but at a smaller additional compute cost (GNN inference is relatively cheap once trained).

11.4 Ablation: Feature and Architecture Contribution

The progression v1 → v2 → v3 shows systematic improvement:

- v1 to v2: RWSE and k-core channels provide stronger structural discrimination.
- v2 to v3: SCC-aware channels, GATv2 architecture with bidirectional messages, global readout, and precision-oriented loss all contribute to improved precision.
- Checkpoint metric change (F1 → topk-precision@8%) directly addresses the false-positive cost in hard-fixing.

Chapter 12

Discussion

12.1 When Exact Algorithms Win

Exact solvers (IC, BST) are the right choice when $n \leq 35$ and $k \leq 15$, as they guarantee optimality. Beyond these ranges, the exponential FPT complexity makes exact methods infeasible within 600-second timeouts. The crossover point between exact and heuristic methods is largely determined by k rather than n, confirming the FPT characterization.

12.2 When Metaheuristics Dominate

For heuristic-track instances ($n = 100$–5000), MA-family solvers provide near-optimal solutions within time budgets. Kernelization (KMA) consistently outperforms plain MA, and higher population sizes improve solution quality at linear time cost.

12.3 Structural Graph Observations

Directed graphs are harder. Directed FVS instances have smaller kernels after SCC reduction, but the remaining kernel is structurally denser and harder for evolutionary search. GNN guidance is more valuable on directed instances, where SCC-aware features (v3) provide structural depth.

Scale-free graphs benefit most from kernelization. Barabási–Albert graphs have high-degree hubs; after removing these hubs (forced by reduction rules), the remaining graph fragments quickly, yielding small kernels.

DAGs are trivially solvable. DAGs have no directed cycles, so their DFVS is empty. Solvers should detect this in $\mathcal{O}(n + m)$ time; the kernelization rules handle this implicitly via the SCC reduction step.

12.4 Practical Implications

The KMA + GNN-KMA v3 combination is recommended for production use when:

- The graph is large (heuristic track) and exact methods are infeasible.
- Trained GNN weights are available for the target graph family.

- A 600-second time budget is available.

Pure KMA with population 90 is recommended when GNN weights are unavailable or the graph family differs significantly from the training distribution.

12.5 Limitations

- **Parameter sensitivity**: KMA and DKMA performance depends on population size, early-stop, and commit threshold. Tuning across graph families would require additional experiments.
- **Large dense graphs**: On dense graphs where kernelization is less effective, all heuristic methods face a harder residual problem. GNN guidance helps less when the kernel is large.
- **GNN training cost**: Generating labeled datasets for v3 training requires running IC/KMA on 200K graphs, which is computationally expensive (∼hours on a workstation).
- **Distribution shift**: GNN models trained on synthetic data may generalize less well to competition instances with different structural properties.
- **PACE gap**: Our best score of 78.93 is significantly below the PACE winner's 99.91, reflecting the sophistication of competition-tuned implementations.

Chapter 13

Future Work

1. **Adaptive kernelization**: Develop data-driven rules that learn which reduction rules are most effective for a given graph family.
2. **Improved GNN architectures**: Explore equivariant GNNs, higher-order message passing, or transformer-based architectures for richer structural representations.
3. **Online learning**: Update GNN weights incrementally as new solved instances become available, enabling continuous improvement.
4. **Distributed MA**: Parallelize population evolution across multiple cores/machines for larger population budgets.
5. **Theoretical bounds**: Prove approximation guarantees for the KMA/DKMA pipeline under specific graph families.
6. **Multi-objective optimization**: Explore Pareto frontiers trading FVS size against runtime or robustness.
7. **PACE 2024/2025**: Apply the framework to future PACE challenge problems to evaluate generality.

Chapter 14

Conclusion

This report presented a comprehensive research and engineering framework for the Feedback Vertex Set problem, spanning theory, exact algorithms, heuristic methods, and graph-neural-network-guided hybrid solvers.

Our primary contributions are:

1. **KMA**: A kernelized memetic algorithm that integrates directed SCC reduction with C++-backed evolutionary search, eliminating 60–90% of directed-graph vertices before search and consistently outperforming plain MA by 4–5 PACE points.
2. **DKMA**: A dynamic re-kernelization loop with SHORTONE contractions, topological diversification, and gain local search, enabling an opening-up effect that progressively shrinks the search space as evolutionary consensus develops.
3. **GNN-KMA v1/v2/v3**: Three generations of GNN-guided hybrid solvers, culminating in v3 with 16-channel SCC-aware features, GATv2 residual blocks, asymmetric precision loss, and topk-precision@8% checkpointing. GNN-KMA v3 achieves 78.64/100 on PACE 2022, comparable to running KMA with 3× more population.
4. **FVS Dataset**: The largest publicly available labeled FVS dataset (200K graphs), enabling reproducible evaluation and future learning-based research.

The framework demonstrates that principled integration of kernelization, evolutionary search, and learned guidance yields competitive solvers while remaining conceptually transparent and extensible. The code, datasets, and weights are publicly available at the project repository, making this work a platform for continued research in combinatorial optimization.

Bibliography

[1] R. M. Karp, "Reducibility among combinatorial problems," in *Complexity of Computer Computations*, 1972, pp. 85–103.

[2] A. Becker and D. Geiger, "Optimization of Pearl's method of conditioning and greedy-like approximation algorithms for the weighted maximum acyclic subgraph problem," *Artificial Intelligence*, vol. 105, pp. 1–2, 1996.

[3] S. Thomassé, "A $4k^2$ kernel for feedback vertex set," *ACM Transactions on Algorithms*, vol. 6, no. 2, 2010.

[4] B. Reed, K. Smith, and A. Vetta, "Finding odd cycle transversals," *Operations Research Letters*, vol. 32, no. 4, pp. 299–301, 2004.

[5] M. Cygan et al., *Parameterized Algorithms*. Springer, 2015.

[6] J. Chen, Y. Liu, S. Lu, B. O'Sullivan, and I. Razgon, "A fixed-parameter algorithm for the directed feedback vertex set problem," *Journal of the ACM*, vol. 55, no. 5, 2008.

[7] T. Lust and J. Teghem, "The multiobjective multidimensional knapsack problem: a survey and a new approach," *International Transactions in Operations Research*, 2010. (Memetic algorithm framework reference.)

[8] G. Grossmann, M. Heuer, P. Schulz, and D. Strash, "Targeted solution methods for the minimum feedback vertex set problem," in *Proc. 17th IPEC*, LIPIcs vol. 249, 2022, doi:10.4230/LIPIcs.IPEC.2022.26.

[9] A. Figiel, V. Froese, A. Nichterlein, and R. Niedermeier, "Improved parameterized algorithms for the directed feedback vertex set problem," in *Proc. ESA 2022*, LIPIcs, 2022, doi:10.4230/LIPIcs.ESA.2022.53.

[10] K. Langedal, J. Langguth, F. Manne, and D. Strash, "Efficient minimum weight vertex cover heuristics using graph neural networks" (Mount-Doom FVS solver), Zenodo, 2022, doi:10.5281/zenodo.6630611.

[11] W. Hamilton, Z. Ying, and J. Leskovec, "Inductive representation learning on large graphs" (GraphSAGE), in *NeurIPS*, 2017.

[12] S. Brody, U. Alon, and E. Yahav, "How attentive are graph attention networks?" (GATv2), in *ICLR*, 2022.

[13] V. P. Dwivedi et al., "Benchmarking graph neural networks," *Journal of Machine Learning Research*, vol. 24, no. 43, 2023.

[14] N. K. Ahmed, J. Neville, R. A. Rossi, and N. Duffield, "Efficient graphlet counting for large networks," in *ICDM*, 2015.

[15] V. Batagelj and M. Zaversnik, "An $\mathcal{O}(m)$ algorithm for cores decomposition of networks," arXiv:cs/0310049, 2003.

[16] R. E. Tarjan, "Depth-first search and linear graph algorithms," *SIAM Journal on Computing*, vol. 1, no. 2, 1972.

[17] A.-L. Barabási and R. Albert, "Emergence of scaling in random networks," *Science*, vol. 286, 1999.

[18] D. J. Watts and S. H. Strogatz, "Collective dynamics of 'small-world' networks," *Nature*, vol. 393, 1998.

[19] P. Erdős and A. Rényi, "On random graphs I," *Publicationes Mathematicae*, vol. 6, 1959.

Appendix A

Directory Structure

```
Feedback_Vertex_Set_Problem_Algorithm_Project_4-2/
|-- cpp_engine/              # C++ exact + heuristic solvers
|-- data/
|    |-- setup_benchmark_inputs.py   # Orchestrator
|    |-- download_real_world.py
|    |-- generate_synthetic.py
|    |-- synthetic/
|    |    |-- undirected/{exact_track,heuristic_track}/{category}/*.txt
|    |    |-- directed/{exact_track,heuristic_track}/{category}/*.txt
|    |    |-- pace2022/            # PACE 2022 instances (protected)
|-- experiments/
|    |-- run_hybrid.py        # KMA/DKMA/GNN-KMA wrappers
|    |-- benchmark_undirected.py
|    |-- benchmark_directed.py
|    |-- run_benchmark_suite.py
|    |-- run_pipeline.py
|-- gnn_model/
|    |-- dataset_gen.py       # TXT -> PT with solver labels
|    |-- train.py             # Variant-aware training
|    |-- model_{undirected,directed}{,_v2}.py
|    |-- model_directed_v3.py
|    |-- feature_engineering_{v2,v3}.py
|    |-- datasets/pt{,_v2,_v3}/  # Variant PT roots
|    |-- weights/             # Trained model weights
|-- build_engine.py
|-- requirements.txt
|-- README.md
```

Listing A.1: Repository directory layout